



9 TYPES OF API TESTING



Test Smarter, Build Better, Deliver Reliable APIs



1



FUNCTIONAL TESTING

Checks whether the API functions work as expected.

```
Example (JavaScript Fetch)

const res = await fetch("https://api.example.com/users/1");
const data = await res.json();

console.assert(res.status === 200);
console.assert(data.id === 1);
```

✓ Verifies correct status and response

2



VALIDATION TESTING

Verifies request/response data and input validation.

```
Example (Invalid Input)

const res = await fetch("/api/users", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ email: "" })
});

console.assert(res.status === 400);
```

✓ Invalid data should be rejected

3



INTEGRATION TESTING

Tests API interactions with databases and other services.

```
Example (Get Order + DB Check)

const res = await fetch("/api/orders/101");
const order = await res.json();

console.assert(order.userId);
console.assert(order.items.length > 0);
```

✓ Ensures dependencies work together

4



LOAD TESTING

Measures API performance under expected traffic.

```
Example (100 Concurrent Requests)

await Promise.all(
  Array.from({ length: 100 }, () =>
    fetch("https://api.example.com/users")
  )
);

console.log("100 requests completed");
```

✓ Handles normal load successfully

5



STRESS TESTING

Tests API behavior under extreme traffic/load.

```
Example (1000 Requests)

const requests = Array.from({ length: 1000 }, () =>
  fetch("https://api.example.com/users")
);

const results = await Promise.allSettled(requests);

console.log(
  results.filter(r => r.status === "fulfilled").length
);
```

✓ Finds breaking point of the API

6



SECURITY TESTING

Finds authentication, authorization, and vulnerability issues.

```
Example (Unauthorized Access)

const res = await fetch("/api/admin", {
  headers: {
    Authorization: "Bearer invalid-token"
  }
});

console.assert(res.status === 401);
```

✓ Unauthorized access should be blocked

7



PERFORMANCE TESTING

Measures response time, throughput, and latency.

```
Example (Response Time)

const start = performance.now();
await fetch("https://api.example.com/products");
const time = performance.now() - start;

console.log(`Response time: ${time.toFixed(2)} ms`);
console.assert(time < 1000);
```

✓ Ensures fast and efficient API

8



RELIABILITY TESTING

Checks API stability over long periods.

```
Example (100 Repeated Requests)

for (let i = 0; i < 100; i++) {
  const res = await fetch("https://api.example.com/health");
  console.assert(res.status === 200);
}

console.log("Reliability test completed");
```

✓ API remains stable over time

9



NEGATIVE TESTING

Tests invalid inputs, missing parameters, and error handling.

```
Example (Invalid Endpoint)

const res = await fetch("/api/users/abc");

console.assert(
  res.status === 400 || res.status === 404
);
```

✓ Proper error handling for bad requests

WHY API TESTING MATTERS?

- ✓ Ensure Reliability & Performance
- ✓ Find Bugs Early
- ✓ Improve Security
- ✓ Better User Experience

THE TESTING JOURNEY



★ PRO TIP

Use tools like Postman, Newman, Jest, Supertest, k6, or JMeter for automation and advanced API testing.



Follow by

m-softtech



Better Testing → Better APIs → Better Applications

